

1. Project Setup

Goal: Initialize the backend application

Tasks:

- Create repository: `tic-tac-toe`
- Set up environment:
 - Python + virtual environment
 - Install dependencies: `fastapi`, `pydantic`

2. Data Storage

Goal: Provide persistence (JSON-based storage)

Tasks:

- Implement:
 - `load_state()` – load JSON on startup
 - `save_state()` – save after each change
- Handle errors:
 - missing file → initialize empty state
- Define structure:
 - players
 - matches
 - statistics

3. Application Management

Goal: Centralize business logic

Tasks:

- Implement `state.py`:
 - keep in-memory state
 - sync with JSON
- Add functions:
 - create player

- delete player
- create match
- update match

4. Game Logic (Tic-Tac-Toe)

Goal: Implement gameplay rules

Tasks:

- Represent board (array of 9 fields)
- Implement:
 - player moves
 - turn switching
 - move validation
- Win detection:
 - use WIN_PATTERNS
- Handle:
 - win
 - draw
 - lose
- Update player statistics

5. API

Goal: Expose backend functionality

Endpoints:

- POST /startMatch
- POST /updateMatch
- POST /createPlayer
- DELETE /deletePlayer
- GET /state

Tasks:

- Implement routes in `routes.py`
- Validate input with Pydantic

6. Dashboard

Goal: Provide system visibility

Tasks:

- Build CLI tool:
 - read JSON
 - display:
 - players
 - stats
 - active matches

7. Validation

Goal: Ensure data survives restart

Tasks:

- Restart app
- Verify:
 - players
 - matches
 - statistics

Result: JSON behaves like a database